

บทที่ 8

การเขียนกริดเซอร์วิส

ในส่วนนี้จะแสดงขั้นตอนการเขียนกริดเซอร์วิสซึ่งได้แบ่งเป็น 5 ขั้นตอน โดยในตัวอย่างนี้จะ เป็น กริดเซอร์วิสทางคณิตศาสตร์อย่างง่าย คือ จะมีคำสั่งบวกและลบ

1. กำหนดรูปแบบของเซอร์วิส ส่วนนี้จะทำโดย GWSDL
2. สร้างเซอร์วิส ส่วนนี้ทำโดยใช้ภาษา Java
3. กำหนดค่าส่วนที่จะนำมาใช้งาน ส่วนนี้จะทำโดย WSDD
4. สร้างไฟล์ชนิด GAR จากการคอมไพล์ทุกส่วนแล้วนำมารวมกัน ส่วนนี้จะใช้ Ant ช่วย
5. นำเซอร์วิสไปไว้ในกริดเซอร์วิสคอนเทนเนอร์ ส่วนนี้จะใช้ Ant ช่วย

8.1 ขั้นตอนที่ 1: กำหนดรูปแบบของเซอร์วิส

ขั้นตอนแรกจะเป็นการเขียนกริดเซอร์วิส หรือ เว็บเซอร์วิส ที่ใช้กำหนดรูปแบบของเซอร์วิส นั้น โดยจะเป็นการกำหนดว่าจะให้เซอร์วิสนี้มีความสามารถอะไรบ้างที่จะให้ผู้อื่นใช้งาน โดยที่ส่วนนี้ จะไม่สนใจว่าจะใช้อัลกอริทึมอย่างไรในการเขียน จะต้องใช้งานไฟล์ไหนบ้าง แต่จะสนใจเพียงว่ามีคำสั่ง อะไรบ้างให้ใช้งาน โดยรูปแบบการใช้งานในกริดเซอร์วิสหรือเว็บเซอร์วิสจะใช้งานผ่านอินเทอร์เฟซที่ เรียกว่า portType ซึ่งส่วนนี้จะกำหนดโดย Web Service Description Language (WSDL)

วิธีการที่จะกำหนด WSDL ที่จะให้ใช้งานในส่วนนี้มีวิธีทำได้อยู่ 2 วิธี ดังต่อไปนี้

- เขียน WSDL ด้วยตนเอง จะสามารถกำหนดทุกอย่างได้ตามที่ตนเองต้องการ แต่ผู้ใช้งานจะไม่คุ้นเคยกับส่วนนี้ เนื่องจาก WSDL เป็นภาษาที่เขียนหลายส่วนมากเกินความจำเป็น
- สร้าง WSDL ขึ้นมาตามรูปแบบของ Java ซึ่งจะสร้างออกมาได้สะดวก แต่อาจไม่ครอบคลุมทุกอย่าง มักมีปัญหาในการทำงานบางอย่าง

โดยตัวอย่างการกำหนดส่วนนี้ทำได้ดังโค้ดตัวอย่างด้านล่างนี้

```
public interface Math
{
    public void add(int a);
    public void subtract(int a);
    public int getValue();
}
```

คราวนี้จะทำให้อยู่ในรูปแบบที่อธิบายด้วย WSDL ที่แม้ว่าจะเข้าใจยากกว่ารูปแบบของ Java แต่การทำงานของมันจะเหมาะสมกับกริดมากกว่า ซึ่งกริดนั้นจะเปลี่ยนแปลงบางส่วนของ WSDL ให้ เป็น GWSDL ซึ่งกำหนดรูปแบบมาจาก OGSi และจาก Globus Toolkit โดย GWSDL นั้นเป็นการขยาย

ส่วนของ WSDL ให้รองรับการอธิบายเกี่ยวกับกริดเซอร์วิส ที่ไม่สามารถอธิบายได้ด้วย WSDL โดยรูปแบบของ GSWDL ที่เหมือนกับที่ได้กำหนดในรูปแบบของ Java ข้างต้นนั้น แสดงได้ดังโค้ดด้านล่างนี้

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions
  name="MathService"

  targetNamespace="http://www.globus.org/namespaces/2004/02/progtutorial/MathService"

  xmlns:tns="http://www.globus.org/namespaces/2004/02/progtutorial/MathService"

  xmlns:ogsi="http://www.gridforum.org/namespaces/2003/03/OGSI"
  "

  xmlns:gwsdl="http://www.gridforum.org/namespaces/2003/03/gridWSDLExtensions"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns="http://schemas.xmlsoap.org/wsdl/">

  <import location="../../../ogsi/ogsi.gwsdl"

  namespace="http://www.gridforum.org/namespaces/2003/03/OGSI"
  />

  <types>
    <xsd:schema
      targetNamespace="http://www.globus.org/namespaces/2004/02/progtutorial/MathService"
      attributeFormDefault="qualified"
      elementFormDefault="qualified"
      xmlns="http://www.w3.org/2001/XMLSchema">
      <xsd:element name="add">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="value" type="xsd:int"/>
          </xsd:sequence>
        </xsd:complexType>
      </xsd:element>
      <xsd:element name="addResponse">
        <xsd:complexType/>
      </xsd:element>
      <xsd:element name="subtract">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="value" type="xsd:int"/>
          </xsd:sequence>
        </xsd:complexType>
      </xsd:element>
      <xsd:element name="subtractResponse">
        <xsd:complexType/>
      </xsd:element>
    </xsd:schema>
  </types>
</definitions>
```

```

<xsd:element name="getValue">
  <xsd:complexType/>
</xsd:element>
<xsd:element name="getValueResponse">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="value" type="xsd:int"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
</xsd:schema>
</types>

<message name="AddInputMessage">
  <part name="parameters" element="tns:add"/>
</message>
<message name="AddOutputMessage">
  <part name="parameters" element="tns:addResponse"/>
</message>
<message name="SubtractInputMessage">
  <part name="parameters" element="tns:subtract"/>
</message>
<message name="SubtractOutputMessage">
  <part name="parameters" element="tns:subtractResponse"/>
</message>
<message name="GetValueInputMessage">
  <part name="parameters" element="tns:getValue"/>
</message>
<message name="GetValueOutputMessage">
  <part name="parameters" element="tns:getValueResponse"/>
</message>

<gwsdl:portType name="MathPortType"
extends="ogsi:GridService">
  <operation name="add">
    <input message="tns:AddInputMessage"/>
    <output message="tns:AddOutputMessage"/>
    <fault name="Fault" message="ogsi:FaultMessage"/>
  </operation>
  <operation name="subtract">
    <input message="tns:SubtractInputMessage"/>
    <output message="tns:SubtractOutputMessage"/>
    <fault name="Fault" message="ogsi:FaultMessage"/>
  </operation>
  <operation name="getValue">
    <input message="tns:GetValueInputMessage"/>
    <output message="tns:GetValueOutputMessage"/>
    <fault name="Fault" message="ogsi:FaultMessage"/>
  </operation>
</gwsdl:portType>
</definitions>

```

หมายเหตุ: โค้ดส่วนที่เน้นตัวอักษรไว้ เป็นส่วนที่สำคัญที่จะนำมาอธิบายเพิ่มเติม ดังต่อไปนี้

ส่วนแรกเป็นเป้าหมายของเนมสเปซจะเป็นอย่างไร

```
http://www.globus.org/namespaces/2004/02/progtutorial/MathService
```

ส่วนที่ 2 เป็นการประกาศเนมสเปซของ OGSi

```
xmlns:ogsi="http://www.gridforum.org/namespaces/2003/03/OGSI"
xmlns:gwsdl="http://www.gridforum.org/namespaces/2003/03/gridWSDLExtensions"
```

ส่วนที่ 3 เป็นการอิมพอร์ต GWSDL และกำหนดชนิดข้อความ และ portType ของ OGSi

```
<import location="../../../ogsi/ogsi.gwsdl"
namespace="http://www.gridforum.org/namespaces/2003/03/OGSI"
/>
```

8.1.1 ส่วนที่แตกต่างกันระหว่าง GWSDL และ WSDL

เมื่อพิจารณาจากโค้ดนี้จะแสดงให้เห็นว่าส่วนที่ขยายขึ้นมาจาก WSDL นั้น คืออะไรบ้าง ดังนี้

```
<gwsdl:portType name="MathPortType"
extends="ogsi:GridService"> </gwsdl:portType>
```

ส่วนแรกที่สำคัญคือ แท็กของ GWSDL ที่แสดงด้วย namespace: <gwsdl:portType> ส่วนนี้จะทำให้สามารถกำหนด portType ให้ขยายได้จาก 1 portType ที่ขยายมาจากหลายๆ portType

ส่วนที่สองที่สำคัญคือ ความเกี่ยวข้องกับ ข้อมูลเซอร์วิส ซึ่งจะแสดงให้เห็นในเรื่องต่อไป

8.2 ขั้นตอนที่ 2: สร้างเซอร์วิส

จากขั้นตอนแรกที่เราได้กล่าวไปเป็นการบอกว่า เซอร์วิสจะทำอะไร ส่วนต่อไปนี้จะเป็นการแสดงว่าจะทำอย่างไรที่กำหนดไว้ในขั้นตอนแรกอย่างไร กล่าวคือ เขียนโปรแกรมให้ทำตามที่กำหนดไว้นั่นเอง ซึ่งใช้ Java ในการเขียนโค้ดในส่วนนี้ โดยจะอธิบายแยกเป็นส่วนๆดังต่อไปนี้

ส่วนแรก เป็นการอิมพอร์ตแพ็คเกจที่ต้องการใช้งาน

```
package org.globus.progtutorial.services.core.first.impl;

import org.globus.ogsa.impl.ogsi.GridServiceImpl;
import
org.globus.progtutorial.stubs.MathService.MathPortType;
import java.rmi.RemoteException;
```

ส่วนนี้จะเป็นการกำหนดคลาสที่ชื่อว่า MathImpl ที่จะใช้สร้างเซอร์วิสที่กำหนดไว้

```
public class MathImpl extends GridServiceImpl implements
MathPortType
```

ส่วนนี้จะเป็นการสร้าง คอนสตรัคเตอร์ ของกริดเซอร์วิส

```
public MathImpl()
{
    super("Simple Math Service");
}
```

ส่วนนี้จะเป็นการกำหนดวิธีการที่จะให้ผลลัพธ์ดังที่กำหนดไว้จากขั้นตอนแรก

```
public void add(int a) throws RemoteException
```

```

{
    value = value + a;
}

public void subtract(int a) throws RemoteException
{
    value = value - a;
}

public int getValue() throws RemoteException
{
    return value;
}

```

เมื่อนำโค้ดแต่ละส่วนมารวมกันแล้ว จะได้โค้ดดังต่อไปนี้

```

package org.globus.progtutorial.services.core.first.impl;

import org.globus.ogsa.impl.ogsi.GridServiceImpl;
import
org.globus.progtutorial.stubs.MathService.MathPortType;
import java.rmi.RemoteException;

public class MathImpl extends GridServiceImpl implements
MathPortType
{
    private int value = 0;

    public MathImpl()
    {
        super("Simple MathService");
    }

    public void add(int a) throws RemoteException
    {
        value = value + a;
    }

    public void subtract(int a) throws RemoteException
    {
        value = value - a;
    }

    public int getValue() throws RemoteException
    {
        return value;
    }
}

```

8.3 ขั้นตอนที่ 3: กำหนดค่าส่วนที่จะนำมาใช้งาน

เมื่อมาถึงขั้นตอนนี้ เราได้ทำส่วนที่สำคัญก็คือ กำหนดรูปแบบของเซอร์วิส และ สร้างเซอร์วิส

มาแล้ว ในขั้นตอนนี้จะนำ 2 ส่วนแรกมารวมเข้าด้วยกัน และทำให้มันเป็นกริดเซอร์วิส ที่จะใช้งานบน Server 1 ได้ ในขั้นตอนนี้มักจะเรียกว่าการ ดีพลอย กริดเซอร์วิส (deployment grid service)

ส่วนที่สำคัญในขั้นตอนนี้คือการกำหนดรายละเอียดการดีพลอยที่เรียกว่า deployment descriptor ซึ่งเป็นไฟล์ที่จะบอก server ว่าจะสร้างกริดเซอร์วิสขึ้นมาอย่างไร (เช่น บอกว่า GSH ของเราจะเป็นอะไร) โดยคำรายละเอียดการดีพลอยนั้น จะเขียนในรูปแบบของ WSDO (Web Service Deployment Descriptor) ซึ่งมีตัวอย่างโค้ดให้ดู ดังนี้

```
<?xml version="1.0"?>
<deployment name="defaultServerConfig"
xmlns="http://xml.apache.org/axis/wsdd/"

xmlns:java="http://xml.apache.org/axis/wsdd/providers/java">

  <service name="progtutorial/core/first/MathService"
provider="Handler" style="wrapped">
    <parameter name="name" value="MathService"/>
    <parameter name="className"
value="org.globus.progtutorial.stubs.MathService.MathPortType"
e"/>

    <parameter name="baseClassName"
value="org.globus.progtutorial.services.core.first.impl.Math
Impl"/>
    <parameter name="schemaPath"
value="schema/progtutorial/MathService/Math_service.wsdl"/>

    <!-- Start common parameters -->
    <parameter name="allowedMethods" value="*" />
    <parameter name="persistent" value="true" />
    <parameter name="handlerClass"
value="org.globus.ogsa.handlers.RPCURIPProvider" />
  </service>
</deployment>
```

จะอธิบายส่วนสำคัญแต่ละส่วน ต่อไปนี้

8.3.1 ชื่อเซอร์วิส

คือโค้ดส่วนนี้

```
<service name="progtutorial/core/first/MathService"
provider="Handler" style="wrapped">
```

เป็นส่วนที่กำหนดว่า กริดเซอร์วิสของเราจะเจอที่ตำแหน่งไหน เราอาจจะนำส่วนนี้มารวมกับที่อยู่ของกริดเซอร์วิสคอนเทนเนอร์ก็ได้ โดยเราจะได้ค่า GSH แบบเต็มจากกริดเซอร์วิส เช่น

ถ้าเราใช้ GT3 ที่มีคอนเทนเนอร์เป็นแบบสแตนด์โตนจะทำให้มีที่อยู่ที่เป็นฐานคือ

<http://localhost:8080/ogsa/services> เมื่อรวมกับใน GSH ของเซอร์วิสที่เรากำหนดไว้แล้ว จะเป็นดังนี้

<http://localhost:8080/ogsa/services/progtutorial/core/first/MathService>

8.3.2 ชื่อเซอร์วิสอีกส่วน

คือโค้ดส่วนนี้

```
<parameter name="name" value="MathService"/>
```

ส่วนนี้จะเป็นส่วนที่บ่งบอกว่าเซอร์วิสนั้นจะมีค่าทั้งเป็น ชื่อแอพลิเคชัน อย่างที่แสดงไว้ส่วนแรก และส่วนนี้จะบ่งบอกถึงพารามิเตอร์ของเซอร์วิส ในโค้ดจะใช้เป็น MathService

8.3.3 className และ baseClassName

คือโค้ดส่วนนี้

```
<parameter name="className"
value="org.globus.progtutorial.stubs.MathService.MathPortType" />
```

```
<parameter name="baseClassName"
value="org.globus.progtutorial.services.core.first.impl.MathImpl" />
```

ใน WSDL นั้น ค่านี้จะบอกว่าคุณได้ใช้สร้างเซอร์วิส (ในตัวอย่างนี้จะใช้ MathImpl จากส่วนก่อนๆ) อย่างไรก็ตามควรมีการแจ้งค่านี้ว่าเป็น Math-PortType stub ที่กำหนดต่อเนื่องมาจากหน้าก่อนๆ

เมื่อกริดเซอร์วิสได้กำหนดให้ยึดหยุ่นมากกว่าเว็บเซอร์วิส จึงจำเป็นต้องกำหนดความแตกต่างกันของ className และ baseClassName โดย className จะอ้างรูปแบบที่แสดงฟังก์ชันของกริดเซอร์วิส (MathPortType) และ baseClassName จะเป็นคลาสที่จัดเตรียมการสร้างกริดเซอร์วิส ในกรณีนี้ baseClassName จะเป็น MathImpl ที่สร้างมาจากส่วนก่อนๆ ในขั้นตอนต่อไป จะแสดงให้เห็นว่า baseClassName ไม่ได้ต้องการรวมส่วนที่สร้างขึ้นมาทั้งหมดของสิ่งที่แสดงใน className

8.3.4 ไฟล์ WSDL

คือโค้ดส่วนนี้

```
<parameter name="schemaPath"
value="schema/progtutorial/MathService/Math_service.wsdl"/>
```

เป็น schemaPath ที่บอกกริดเซอร์วิสคอนเทนเนอร์ว่า WSDL ไฟล์นั้นจะสามารถเจอได้ที่ไหน ที่ไม่ได้บอกถึง GWSDL เนื่องจาก GWSDL ไม่ใช่มาตรฐาน แต่เป็นส่วนที่ขยายมาจาก WSDL โดยขั้นแรกจะต้องแปลงให้เป็น WSDL เพื่อใช้บอกว่าการใช้งานกับเทคโนโลยีเว็บเซอร์วิสที่มีอยู่จริง ซึ่งไฟล์ WSDL นั้นจะสร้างโดยอัตโนมัติด้วย GT3 เมื่อเราคอมไพล์เซอร์วิส

8.3.5 พารามิเตอร์ทั่วไป

คือโค้ดส่วนนี้ ซึ่งเป็นค่าพารามิเตอร์ที่ใช้ในโปรแกรมตัวอย่าง

```
<!-- Start common parameters -->
```

```
<parameter name="allowedMethods" value="*" />
```

```
<parameter name="persistent" value="true" />
```

```
<parameter name="handlerClass"
value="org.globus.ogsa.handlers.RPCURIProvider" />
```

8.4 ขั้นตอนที่ 4: สร้างไฟล์ชนิด GAR จากการคอมไพล์ทุกส่วนแล้วนำมารวมกัน

เมื่อมาถึงขั้นตอนนี้เราจะได้ รูปแบบของเซอร์วิสจากขั้นที่ 1 ได้เซอร์วิสที่สร้างขึ้นจากขั้นที่ 2 และได้ส่วนที่บอกกริดเซอร์วิสคอนเทนเนอร์ว่าจะแสดงส่วนที่ได้จากขั้นที่ 1 และ 2 ไปให้ภายนอกเห็น อย่างไรก็ตาม แต่ส่วนที่ได้มานั้นยังไม่สามารถใช้งานได้จริงในคอนเทนเนอร์ เนื่องจากไฟล์ Java ก็ยังไม่ได้คอมไพล์และไฟล์อื่นๆก็ยังไม่รู้ว่าต้องนำไปวางที่ส่วนไหน

ดังนั้นในขั้นนี้เราจะรวมทุกส่วนที่กล่าวมาเข้าด้วยกัน เพื่อให้นำไปใช้งานได้จริงได้ โดยเราจะสร้างไฟล์ Grid Archive หรือไฟล์ประเภท GAR ขึ้นมา โดยที่ไฟล์ Gar นั้นจะเป็นไฟล์เดี่ยวที่บรรจุไฟล์ทั้งหมดและข้อมูลที่กริดเซอร์วิสคอนเทนเนอร์ต้องการทั้งหมดไว้เพื่อที่จะดีพลอยไปให้แก่เซอร์วิสที่เราสร้างขึ้นให้ทำงานออกมาใช้งาน ต่อมาเราจะแสดงวิธีที่จะทำไฟล์ GAR และดีพลอยมัน โดยขั้นตอนนี้จะแบ่งออกได้ดังนี้

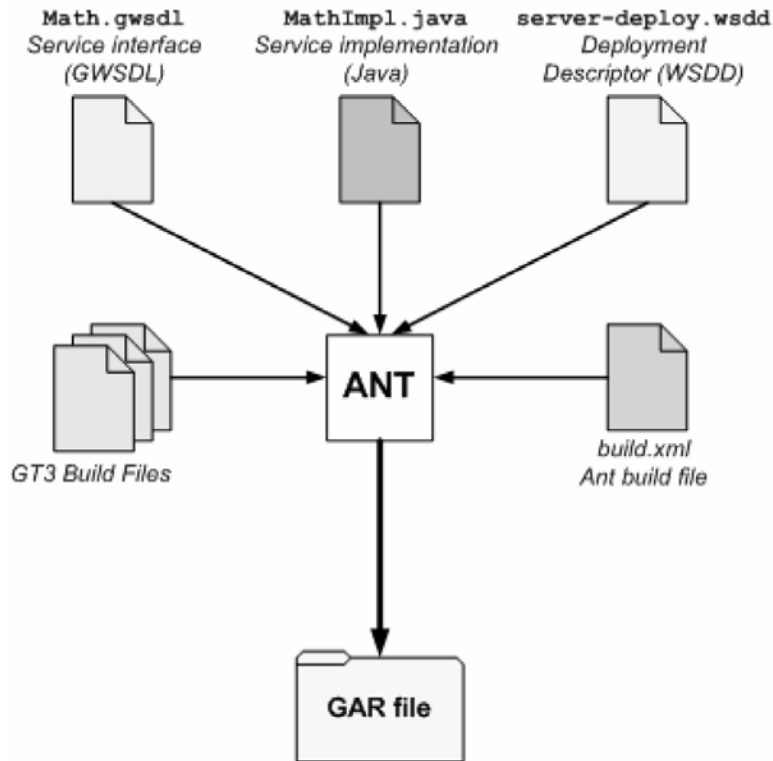
- แปลงจาก GWSDL ให้เข้าสู่รูปแบบมาตรฐานคือ WSDL
- สร้างสแต็คคลาสจาก WSDL
- คอมไพล์สแต็คคลาสนั้นๆ
- คอมไพล์เซอร์วิสที่เราสร้างขึ้น
- รวบรวมไฟล์ทั้งหมดเข้าสู่โครงสร้างที่จะใช้โดยเฉพาะ

ขั้นตอนเหล่านี้ เมื่อกล่าวขึ้นมาอาจจะฟังดูยาก แต่ Globus ก็ได้มีเครื่องมือที่ช่วยในขั้นตอนนี้ นั่นก็คือ Ant ที่สามารถทำขั้นตอนที่กล่าวมาได้ในขั้นตอนเดียว

8.4.1 Ant

Ant เป็น ซอฟต์แวร์ของ Apache ที่เป็นเครื่องมือสร้าง Java โดยอาศัยหลักการคล้ายคำสั่ง make ของ UNIX ซึ่งทำให้คนเขียนโปรแกรมสามารถข้ามขั้นตอนที่ยากเกี่ยวกับการนำไฟล์เอ็กคิวต์จากไฟล์ต้นทาง ซึ่งส่วนนี้จะทำโดย Ant ที่จะช่วยลดขั้นตอนในส่วนนี้ให้เหลือขั้นเดียว ซึ่งด้วย Ant นี้จะทำให้ปัญหาของการทำกริดเซอร์วิสนั้นมาอยู่ที่การกำหนดรูปแบบและการสร้างมันขึ้นมา

ซึ่ง Ant ต้องการไฟล์ที่ใช้สร้างอยู่ 2 ไฟล์ คือ ไฟล์ที่ใช้สร้างที่เป็นส่วนหนึ่งของ GT3 และอีกส่วนก็จะเป็นส่วนที่สำคัญที่เราจะต้องเขียนขึ้นมาเอง (ตามขั้นตอนที่สร้างโค้ด WSDL และสร้างสแต็ค) เมื่อสร้างไฟล์นี้ขึ้นมาแล้วอาจจะใช้ได้กับหลายๆกริดเซอร์วิสโดยไม่ต้องสร้างขึ้นมาใหม่ ซึ่งส่วนนี้ Ant ก็ได้ทำไว้ให้แล้ว แต่ถ้าต้องการงานที่ยืดหยุ่นมากขึ้นก็จะต้องสร้างไฟล์นี้ขึ้นมาใหม่ โดยเราจะแสดงองค์ประกอบของส่วนนี้ให้เห็นดังรูปต่อไปนี้



รูปที่ 8-1 แสดงองค์ประกอบในการใช้งาน Ant

สิ่งที่จำเป็นต้องทำในขั้นตอนนี้

จำเป็นต้องใส่ไดเรกทอรีที่เราลง GT3 ไว้ ดังในตัวอย่างข้างล่างนี้

ogsa.root=/usr/local/gt3

สร้างเซอร์วิสที่เป็น GAR (ตัวอย่างนี้จะสร้าง MathService) โดยใส่ สคริปต์ ตามนี้

./tutorial_build.sh <service base directory> <service's GWSDL file>

โดยที่ <service base directory> นั้นเป็นไดเรกทอรีที่เราวาง server-deploy.wsdd ไว้และกำหนด
ที่ที่จะเจอ MathImpl.java ดังตัวอย่างข้างล่างนี้

**./tutorial_build.sh\org/globus/progtutorial/services/core/
first
\schema/progtutorial/MathService/Math.gwsdl**

ถ้าทุกขั้นตอนนั้นถูกต้องไฟล์ GAR นั้นจะอยู่ในตำแหน่งของ \$TUTORIAL_DIR/build/lib. ดัง
ในตัวอย่างนี้

**\$TUTORIAL_DIR/build/lib/org_globus_progtutorial_services_cor
e_first.gar**

8.5 ขั้นตอนที่ 5: นำเซอร์วิสไปไว้ในกริดเซอร์วิสคอนเทนเนอร์

ไฟล์ GAR จากขั้นตอนที่แล้วนั้นจะบรรจุไฟล์ที่จำเป็นที่จะใช้ในกริดเซอร์วิสแล้ว ขั้นนี้เราดี

พลอยมันโดย Ant ซึ่งจะนำไฟล์เหล่านี้ คือ WSDL, สดับที่คอมไพล์แล้ว, โค้ดที่สร้างขึ้นคอมไพล์แล้ว และที่ WSDD ไปไว้ที่ตำแหน่งที่ให้ใช้งานที่กำหนดไว้ใน GT3 ซึ่งคำสั่งเกี่ยวกับการดีพลอยนั้นเราจะต้องทำที่ root ของส่วนที่เราลง GT3 โดยใช้คำสั่งตามรูปแบบนี้

```
ant deploy -Dgar.name=<full path of GAR file>
```

สำหรับในตัวอย่างนี้จะได้โค้ดดังนี้

```
ant deploy \ -Dgar.name=$TUTORIAL_DIR/build/lib/  
org_globus_progtutorial_services_core_first.gar
```

การดีพลอยในขั้นตอนสุดท้ายนี้ก็ทำแค่เพียงเท่านี้ ซึ่งเมื่อมาถึงขั้นนี้เราจะได้ 5 ขั้นตอนที่เป็นของการทำกริดเซอร์วิสแล้ว ต่อมาเราจะมาแสดงขั้นตอนในการเขียนแอปพลิเคชันในฝั่ง client

8.6 แอปพลิเคชันฝั่งไคลเอนท์

เมื่อเราต้องการทดสอบกริดเซอร์วิสที่เขียนขึ้นด้วยคอมมานด์-ไลน์ ซึ่งจะใช้วิธี `getValue` ในการทดสอบ `MathService` นี้ สิ่งที่ client จะต้องได้รับจากคอมมานด์-ไลน์มี 2 คำคือ

- GSH
- ค่าที่เพิ่มขึ้นตามการทำงานที่กำหนดไว้ในเซอร์วิส

เราใช้โค้ดนี้ในการทดสอบการทำงานของ `MathService`

```
package org.globus.progtutorial.clients.MathService;  
  
import  
org.globus.progtutorial.stubs.MathService.service.MathService  
eGridLocator;  
import  
org.globus.progtutorial.stubs.MathService.MathPortType;  
  
import java.net.URL;  
  
public class Client  
{  
    public static void main(String[] args)  
    {  
        try  
        {  
            // Get command-line arguments  
            URL GSH = new java.net.URL(args[0]);  
            int a = Integer.parseInt(args[1]);  
  
            // Get a reference to the MathService instance  
            MathServiceGridLocator mathServiceLocator = new  
MathServiceGridLocator();
```

```

        MathPortType math =
        mathServiceLocator.getMathServicePort(GSH);

        // Call remote method 'add'
        math.add(a);
        System.out.println("Added " + a);

        // Get current value through remote method 'getValue'
        int value = math.getValue();
        System.out.println("Current value: " + value);
    }catch(Exception e)
    {
        System.out.println("ERROR!");
        e.printStackTrace();
    }
}
}

```

แต่ก่อนที่เราจะรันคอมไพเลอร์นี้ สิ่งที่ต้องรันก่อน มีดังต่อไปนี้

```
source $GLOBUS_LOCATION/etc/globus-devel-env.sh
```

เพื่อที่จะคอมไพล์ client จะต้องทำตามนี้

```
javac \ -classpath ./build/classes/:$CLASSPATH \
org/globus/progtutorial/clients/MathService/Client.java
```

โดยก่อนที่จะให้มันทำงานเราต้องเริ่มการทำงานของคอนเทนเนอร์ก่อน โดยรันคำสั่งนี้ที่ root

```
globus-start-container
```

เมื่อคอนเทนเนอร์ทำงานแล้ว เราจะเห็นรายการของ GSH ที่มีขึ้นมา ซึ่งเราต้องเลือก MathService ตามนี้

```
http://127.0.0.1:8080/ogsa/services/progtutorial/core/first/
MathService
```

คราวนี้จะเป็นการสรุปคำสั่งเต็มๆที่จะให้ client ทำงาน

```
java \
-classpath ./build/classes/:$CLASSPATH \
org.globus.progtutorial.clients.MathService.Client \
http://127.0.0.1:8080/ogsa/services/progtutorial/core/first/
MathService \
5
```

ถ้าคำสั่งนี้ผ่านเราจะเห็นค่าต่อไปนี้

```
Added 5
```

```
Current value: 5
```

ซึ่งแสดงว่าคำสั่งบวกของเราใช้งานได้แล้ว โดยจะเริ่มบวกค่าจากค่าเริ่มต้นที่เป็น 0 ทำให้ได้ค่า 5 ขึ้นมา เมื่อเรารันคำสั่งนั้นอีกครั้ง เราจะได้ค่าดังต่อไปนี้

```
Added 5
```

```
Current value: 10
```

ซึ่งแสดงว่าสเตทภายในของมันนั้นยังเก็บค่า 5 ซึ่งเป็นค่าเดิมไว้อยู่ ทำให้จะได้ค่าใหม่ออกมาเป็น 10 ซึ่งเพิ่มจากค่าเดิมนั่นเอง